Computer Vision
Lecture 14: Convolution

Yeungnam University
MSP lab.
Ho-Youl JUNG

LTI (Linear Time Invariant) system

$$y(n) = T\{x(n)\}$$

$$= T\{\sum_{k=-\infty}^{\infty} x(k)\delta(n-k)\}$$

$$= \sum_{k=-\infty}^{\infty} x(k) T\{\delta(n-k)\}$$

$$= \sum_{k=-\infty}^{\infty} x(k) h(n-k)$$

$$= x(n) * h(n)$$



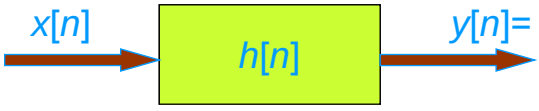
Linear system:

T $\{a_1x_1[n]+a_2x_2[n]\}=a_1T\{x_1[n]\}+a_2\,T\{x_2[n]\}$

Time invariant system

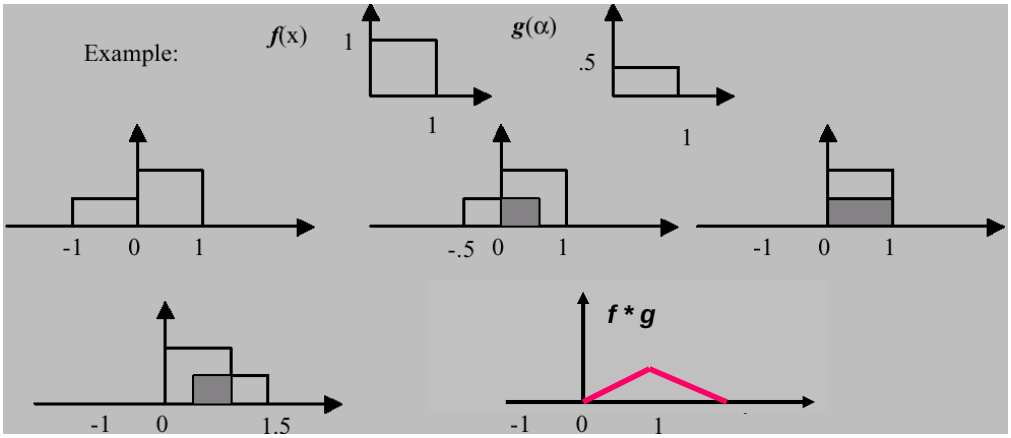
Impulse (unit sample) response

Discrete convolution



Convolution (1-D continuous)

$$f(x) * g(x) = \int_{-\infty}^{\infty} f(\alpha) g(x-\alpha) d\alpha$$

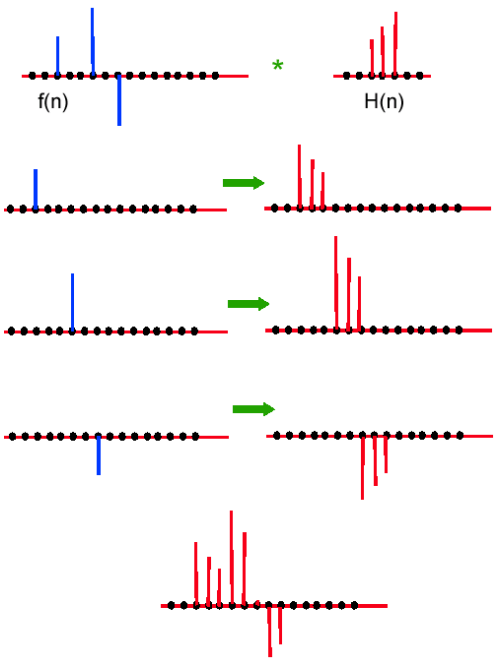


$$f(x) * g(x) \Leftrightarrow F(u)G(u)$$
$$f(x)g(x) \Leftrightarrow F(u) * G(u)$$

1-D Convolution :

$$f(n) * h(n) = \sum_{m=0}^{M-1} f(m)h(n-m)$$

$$h(n) * f(n) = \sum_{m=0}^{M-1} h(m)f(n-m)$$



1-D Convolution

$$f(n) * h(n) = \sum_{m=0}^{M-1} f(m)h(n-m)$$

$$h(n) * f(n) = \sum_{m=0}^{M-1} h(m)f(n-m)$$

h(n)
3 tap filter

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)			1	2	0	0	−1	−2			

n=1

h(1−m)	1	2	3

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)				5							

1-D Convolution

$$f(n) * h(n) = \sum_{m=0}^{M-1} f(m)h(n-m)$$

$$h(n) * f(n) = \sum_{m=0}^{M-1} h(m)f(n-m)$$

h(n)
3 tap filter

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)			1	2	0	0	−1	−2			

n=2

h(2−m)	1	2	3

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)				5	2						

1-D Convolution

$$f(n) * h(n) = \sum_{m=0}^{M-1} f(m)h(n-m)$$

$$h(n) * f(n) = \sum_{m=0}^{M-1} h(m)f(n-m)$$

h(n)
3 tap filter

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)			1	2	0	0	−1	−2			

n=3

h(2−m)	1	2	3

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)				5	2	−3	−8				

1-D Convolution

$$f(n) * h(n) = \sum_{m=0}^{M-1} f(m)h(n-m)$$

$$h(n) * f(n) = \sum_{m=0}^{M-1} h(m)f(n-m)$$

h(n)
3 tap filter

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)			1	2	0	0	−1	−2			

n=4

h(4−m)	1	2	3

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)				5	2	−3	−8				

1-D Convolution : zero padding

h(n)
3 tap filter

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)		0	1	2	0	0	−1	−2	0		

n=0

h(m)	1	2	3

Zero padding

n=5

h(m)	1	2	3

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)			8	5	2	−3	−8	−5			

1-D Convolution : circular convolution

h(n)

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)		−2	1	2	0	0	−1	−2	1		

n=0

h(m)	1	2	3

Circular

n=5

h(m)	1	2	3

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)			6	5	2	−3	−8	−2			

1-D Convolution : circular convolution

h(n)

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)		−2	1	2	0	0	−1	−2	1		

```
for( i=0; i<6; i++){
    y[i] = f[i-1] + 2 * f[i] + 3* f[i+1] ;
}
```

Circular Convolution

```
For(i=0; i<6; i++){
    y[i] = f[(i-1+6)%6] + 2 * f[I] + 3 *   f[(i+1)%6] ;
}
```

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)			6	5	2	−3	−8	−2			

1-D Convolution : circular convolution

h(n)

n	−1	0	1
h(n)	3	2	1

f(n)=f(m)

n	−2	−1	0	1	2	3	4	5	6	7	8
f(n)		−2	1	2	0	0	−1	−2	1		

$$f(n) * h(n) = [1\ 2\ 0\ 0\ -1\ -2] * [3\ 2\ 1]$$
$$= [6\ 5\ 2\ -3\ -8\ -2]$$

Circular Convolution

$$\begin{bmatrix} 2 & 3 & 0 & 0 & 0 & 1 \\ 1 & 2 & 3 & 0 & 0 & 0 \\ 0 & 1 & 2 & 3 & 0 & 0 \\ 0 & 0 & 1 & 2 & 3 & 0 \\ 0 & 0 & 0 & 1 & 2 & 3 \\ 3 & 0 & 0 & 0 & 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 0 \\ 0 \\ -1 \\ -2 \end{bmatrix} = \begin{bmatrix} 6 \\ 5 \\ 2 \\ -3 \\ -8 \\ -2 \end{bmatrix}$$

y(n)

n	−2	−1	0	1	2	3	4	5	6	7	8
y(n)			6	5	2	−3	−8	−2			

1-D Convolution : zero padding

h(n)

n	−1	0	1
h(n)	3	2	1

n	f(n)
−2	
−1	0
0	1
1	2
2	0
3	0
4	−1
5	−2
6	0
7	
8	

h(−m)
1
2
3

y(n)
8
5
2
−3
−8
−5

1-D Convolution : zero padding

h(n)

n	−1	0	1
h(n)	3	2	1

n	f(n)
−2	
−1	0
0	1
1	2
2	0
3	0
4	−1
5	−2
6	0
7	
8	

y(n)
8
5
2
−3
−8
−5

i

2

3

y(2)=1×f(1)+2×f(2)+3×f(3)

y(3)=1×f(2)+2×f(3)+3×f(4)

y(4)=1×f(3)+2×f(4)+3×f(5)

1-D Convolution : Successive two filters

h(n)

n	−1	0	1
h(n)	3	2	1

g(n)

n	−1	0	1
g(n)	1	2	3

n	f(n)
−2	
−1	0
0	1
1	2
2	0
3	0
4	−1
5	−2
6	0
7	
8	

y(n)
0
8
5
2
−3
−8
−5
0

$y[n] = h[n] * f[n]$

z(n)
21
36
16
−8
−30

$z[n] = g[n] * y[n]$
$= g[n] * (h[n] * f[n])$

y(2)=1×f(1)+2×f(2)+3×f(3)

y(3)=1×f(2)+2×f(3)+3×f(4)

y(4)=1×f(3)+2×f(4)+3×f(5)

z(n)=[f(n)*h(n)]*g(n)

z(3)=3×y(2)+2×y(3)+1×y(4)

=3×[1×f(1)+2×f(2)+3×f(3)]

+2×[1×f(2)+2×f(3)+3×f(4)]

+1×[1×f(3)+2×f(4)+3×f(5)]

=3× f(1) +8×f(2) +14×f(3)

+ 8×f(4)+ 3×f(5)

1-D Convolution : Successive two filters

h(n)

n	−1	0	1
h(n)	3	2	1

g(n)

n	−1	0	1
g(n)	1	2	3

n	f(n)
−2	
−1	0
0	1
1	2
2	0
3	0
4	−1
5	−2
6	0
7	
8	

z(n)
21
36
16
−8
−30

$z[n] = g[n] * y[n]$
$= (g[n] * h[n]) * f[n]$

Receptive field :
3+3-1=5

z(n)=[f(n)*h(n)]*g(n)

z(3)=3×y(2)+2×y(3)+1×y(4)

=3×[1×f(1)+2×f(2)+3×f(3)]

+2×[1×f(2)+2×f(3)+3×f(4)]

+1×[1×f(3)+2×f(4)+3×f(5)]

=3× f(1) +8×f(2) +14×f(3)

+ 8×f(4)+ 3×f(5)

1-D Convolution : Properties

- Communicative property

$$f_1(n) * f_2(n) = f_2(n) * f_1(n)$$

- Distributive property

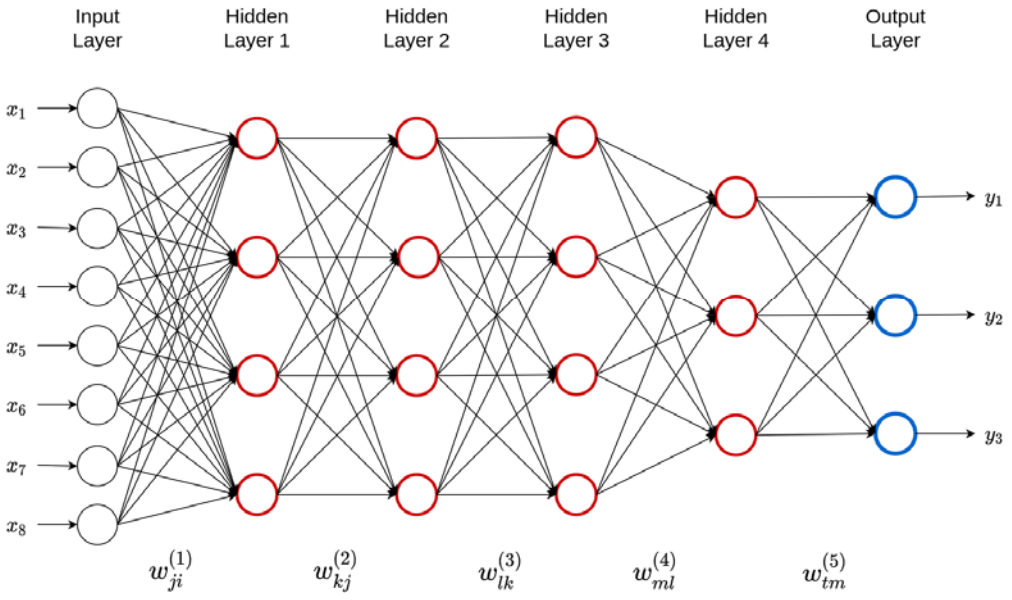
$$f_1(n) * [f_2(n) + f_3(n)] = f_1(n) * f_2(n) + f_1(n) * f_3(n)$$

- Associative property

$$f_1(n) * [f_2(n) * f_3(n)] = [f_1(n) * f_2(n)] * f_3(n)$$

Fully Connected Network

- A five-layer feedforward deep neural network with one input layer, four hidden layers and one output layer
- Activation function(sigmoid, softmax, relu..) 가 없으면 linear filter
- Training : 적합한 filter 의 계수를 구하는 과정



https://media.springernature.com/full/springer-static/image/art%3A10.1186%2Fs12864-020-07319-x/MediaObjects/12864_2020_7319_Fig1_HTML.png?as=webp

2-D Convolution

$$f(x,y) * h(x,y) = \int_{-\infty}^{+\infty}\int f(\alpha,\beta)h(x-\alpha, y-\beta)d\alpha\, d\beta$$

Continuous

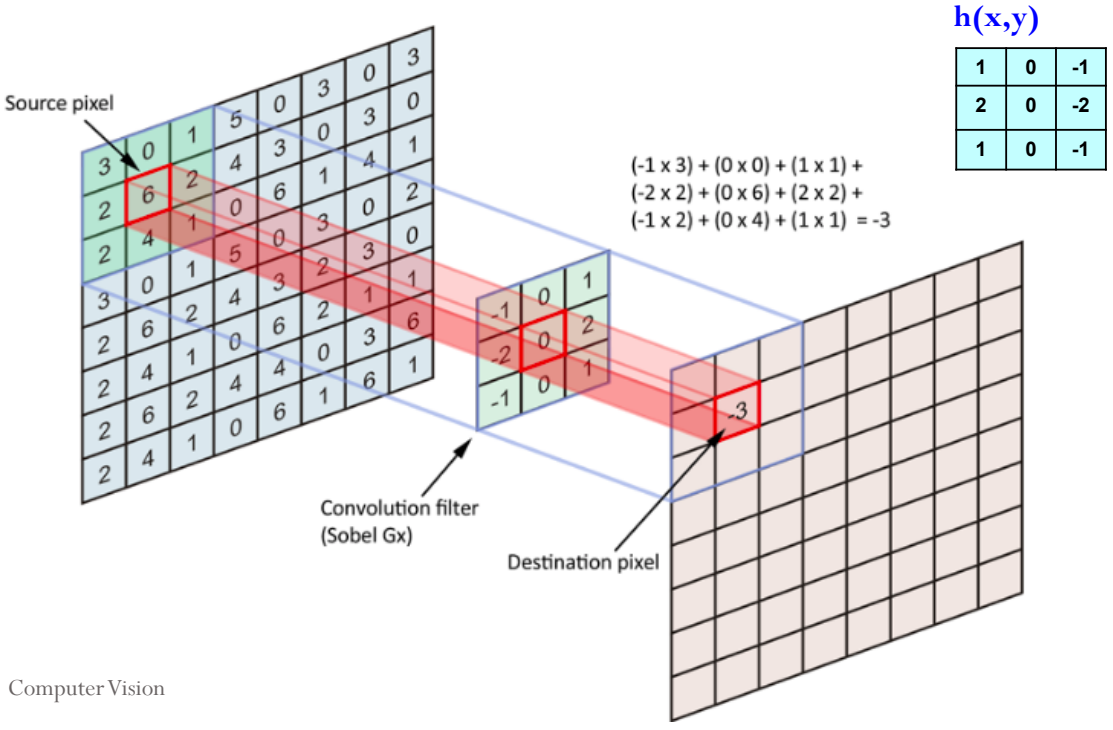
$$f(x,y) * h(x,y) \Leftrightarrow F(u,v)H(u,v)$$
$$f(x,y)h(x,y) \Leftrightarrow F(u,v) * H(u,v)$$

Discrete

$$f(x,y) * h(x,y) = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f(m,n)h(x-m, y-n)$$

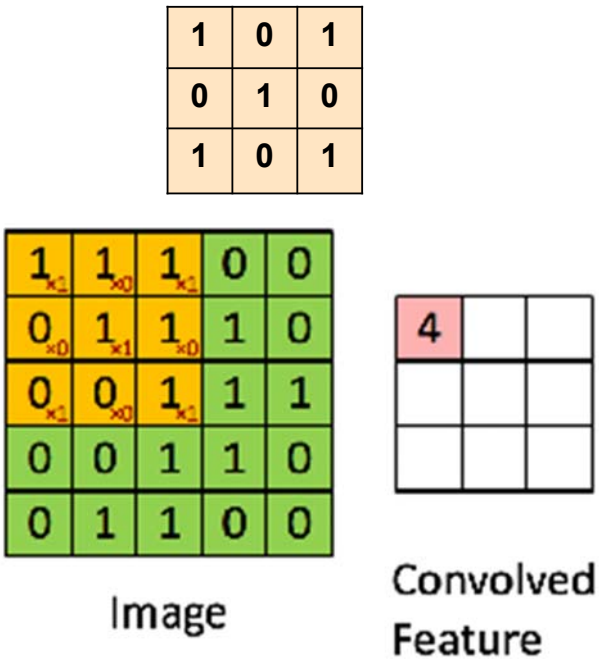
2-D Convolution

$$f(x,y) * h(x,y) = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f(m,n)h(x-m, y-n)$$



2-D Convolution

- Stride 1



Source: *https://ujwlkarn.files.wordpress.com/2016/07/convolution_schematic.gif?w=268&h=196*

2-D Convolution

- Stride 1
- Zero padding

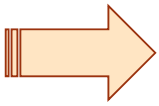
h(m,n)

1	0	-1
2	0	-2
1	0	-1

h(-s,-t)

-1	0	1
-2	0	2
-1	0	1

0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3	0	3	0
0	2	5	2	4	3	0	3	0	0
0	2	4	1	0	6	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0



5	-4	9	-1	-8	0	0	-3
14	-3						

2-D Convolution

- Stride 1
- Zero padding

h(m,n)

1	0	-1
2	0	-2
1	0	-1

h(-s,-t)

-1	0	1
-2	0	2
-1	0	1

0	0	0	0	0	0	0	0	0	0
0	3	0 -1	1 0	5 1	0	3	0	3	0
0	2	5 -2	2 0	4 2	3	0	3	0	0
0	2	4 -1	1 0	0 1	6	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

5	-4	9	-1	-8	0	0	-3
14	-3	-1					

2-D Convolution

- Stride 1

h(m,n)

1	0	-1
2	0	-2
1	0	-1

h(-s,-t)

-1	0	1
-2	0	2
-1	0	1

0	0	0	0	0	0	0	0	0	0
0	3	0	1 -1	5 0	0 1	3	0	3	0
0	2	5	2 -2	4 0	3 2	0	3	0	0
0	2	4	1 -1	0 0	6 1	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

5	-4	9	-1	-8	0	0	-3
14	-3	-1	6				

2-D Convolution

- Stride 1
- Zero padding

h(m,n)

1	0	-1
2	0	-2
1	0	-1

h(-s,-t)

-1	0	1
-2	0	2
-1	0	1

0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3_{-1}	0_{0}	3_{1}	0
0	2	5	2	4	3	0_{-2}	3_{0}	0_{2}	0
0	2	4	1	0	6	1_{-1}	4_{0}	1_{1}	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

5	-4	9	-1	-8	0	0	-3
14	-3	-1	6	-9	-2	□	-□

2-D Convolution

- Stride 1
- Zero padding

h(m,n)

1	0	-1
2	0	-2
1	0	-1

h(-s,-t)

-1	0	1
-2	0	2
-1	0	1

0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3	0	3	0
0_{-1}	2_{0}	5_{1}	2	4	3	0	3	0	0
0_{-2}	2_{0}	4_{2}	1	0	6	1	4	1	0
0_{-1}	3_{0}	0_{1}	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

5	-4	9	-1	-8	0	0	-3
14	-3	-1	6	-9	-2	□	-□
13	-4						

2-D Convolution

- Stride 1
- Zero padding

h(m,n)

1	0	-1
2	0	-2
1	0	-1

h(-s,-t)

-1	0	1
-2	0	2
-1	0	1

0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3	0	3	0
0	2	5	2	4	3	0	3	0	0
0	2	4	1	0	6	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

5	-4	9	-1	-8	0	0	-3
14	-3	-1	6	-9	-2	[illegible]	-10
13	-4	-4	10	-4	-4	-1	-11
10	-5	4	4	-5	-2	-4	-7
16	-3	-3	6	-4	-5	-6	-7
20	-2	-12	13	-2	-11	1	-8
20	-2	-12	14	-5	-7	11	-13
14	-2	-10	12	-2	-1	6	-15

2-D Convolution

- sub-sampling by 2
- Zero padding

0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3	0	3	0
0	2	5	2	4	3	0	3	0	0
0	2	4	1	0	6	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

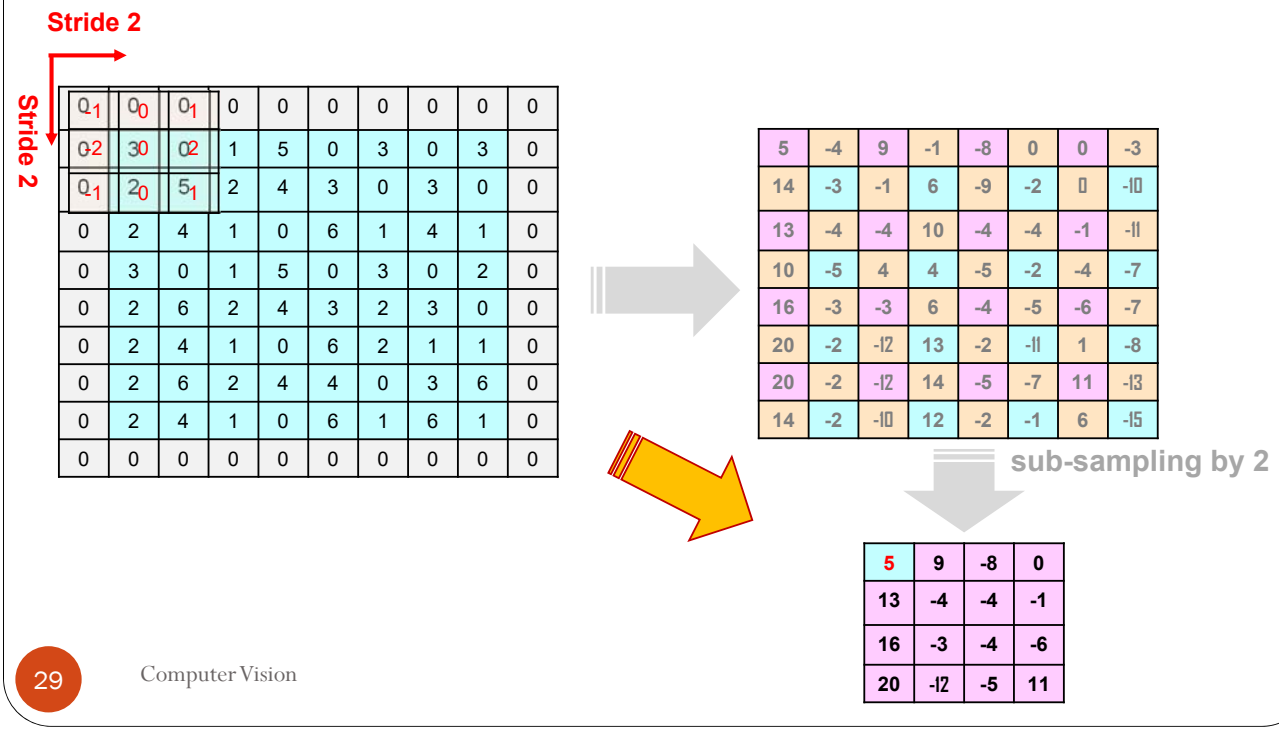
5	-4	9	-1	-8	0	0	-3
14	-3	-1	6	-9	-2	[illegible]	-10
13	-4	-4	10	-4	-4	-1	-11
10	-5	4	4	-5	-2	-4	-7
16	-3	-3	6	-4	-5	-6	-7
20	-2	-12	13	-2	-11	1	-8
20	-2	-12	14	-5	-7	11	-13
14	-2	-10	12	-2	-1	6	-15

sub-sampling by 2

5	9	-8	0
13	-4	-4	-1
16	-3	-4	-6
20	-12	-5	11

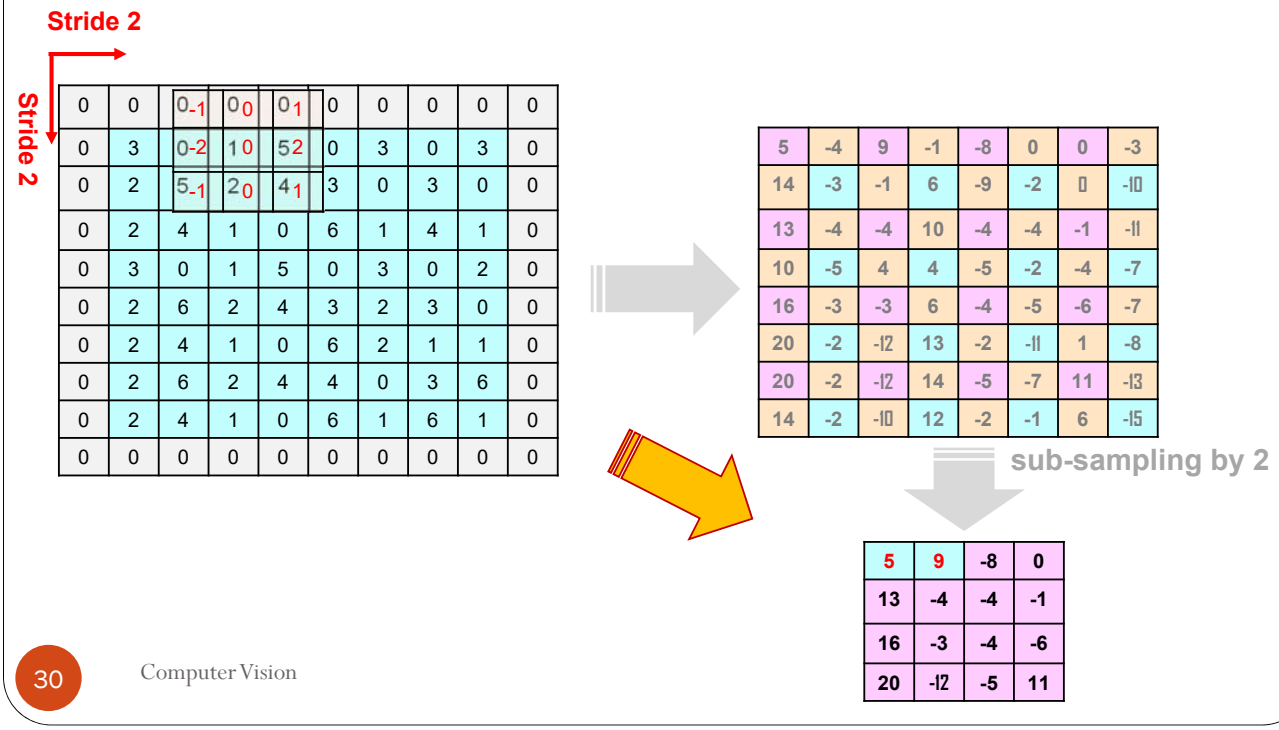
2-D Convolution

- Stride 2 = sub-sampling by 2
- Zero padding



2-D Convolution

- Stride 2 = sub-sampling by 2
- Zero padding



2-D Convolution

- Max pooling with 2x2 and stride 2
- Zero padding

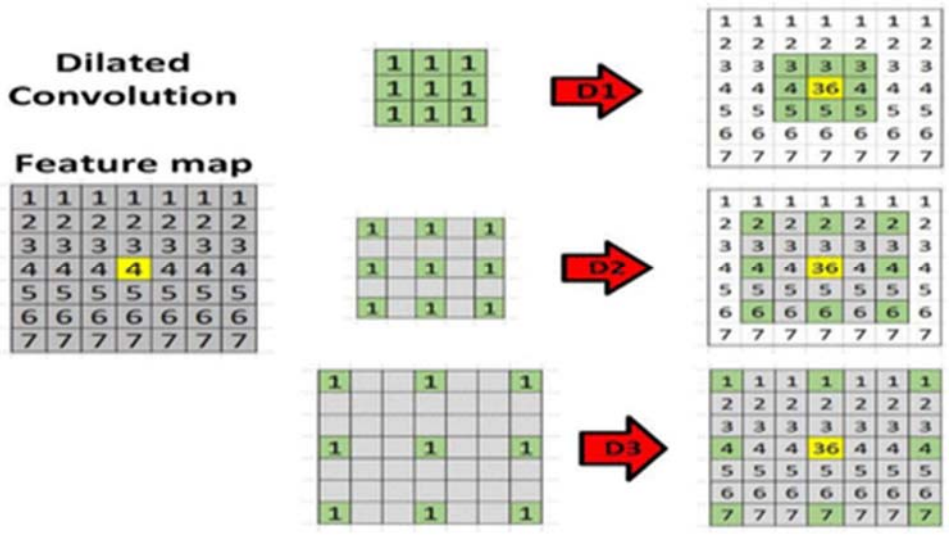
0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3	0	3	0
0	2	5	2	4	3	0	3	0	0
0	2	4	1	0	6	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

5	-4	9	-1	-8	0	0	-3
14	-3	-1	6	-9	-2	0	-10
13	-4	-4	10	-4	-4	-1	-11
10	-5	4	4	-5	-2	-4	-7
16	-3	-3	6	-4	-5	-6	-7
20	-2	-12	13	-2	-11	1	-8
20	-2	-12	14	-5	-7	11	-13
14	-2	-10	12	-2	-1	6	-15

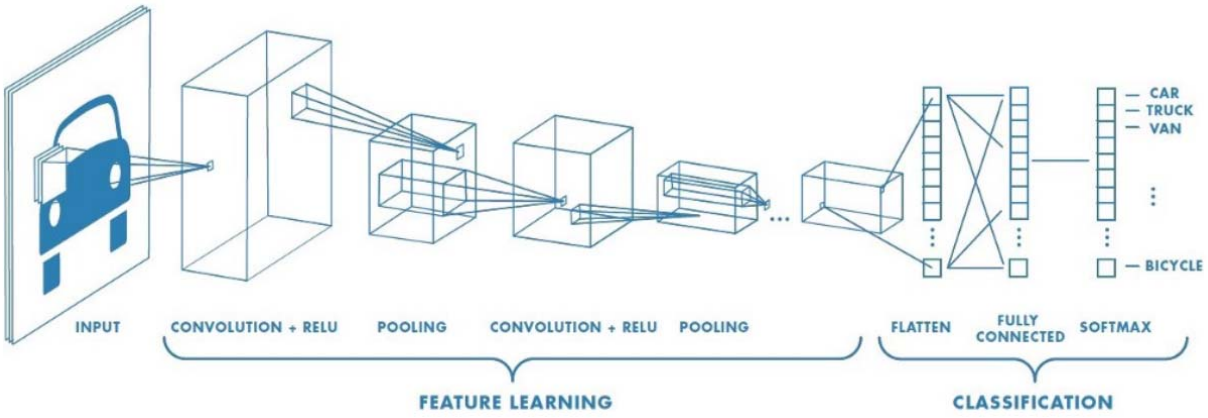
Max pooling
with 2x2 and stride 2

14	9	0	0
13	10	-2	-1
20	14	-2	1
20	14	-1	11

2-D Convolution : Dilated convolution



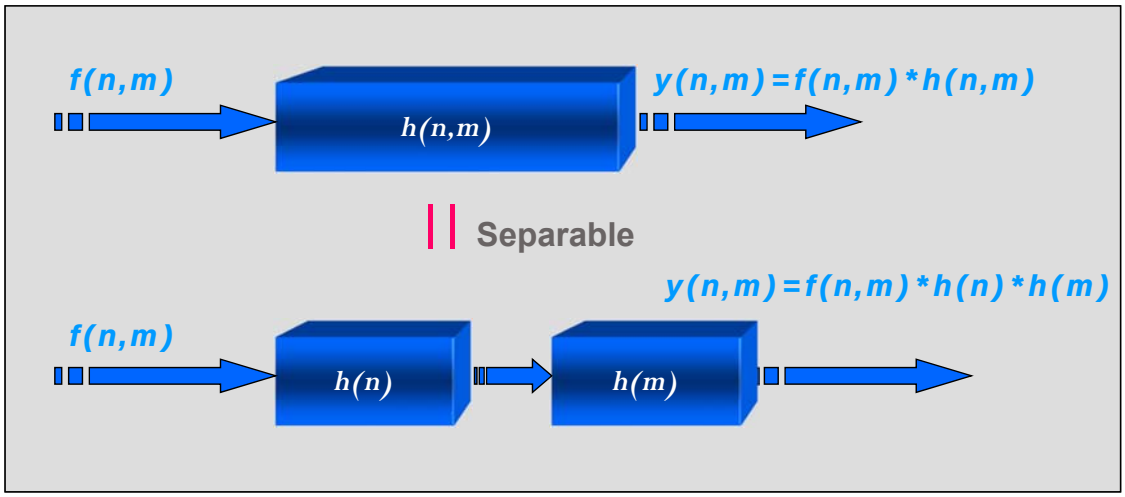
Example of CNN(Convolutional Neural Network)



https://miro.medium.com/max/1000/1*vkQ0hXDaQv57sALXAJquxA.jpeg

2-D Convolution : separable filter

- Separable vs Non-Separable Filter



*If h(n,m)=h(n)*h(m), h(n,m) is separable*

2-D Convolution : separable filter

- If h(m,n) = h(m) * h(n), h(m,n) is separable

h(m,n)

1	0	-1
2	0	-2
1	0	-1

1	0	-1

1
2
1

h(-s)

-1	0	1

h(-t)

1
2
1

0	0	0	0	0	0	0	0	0	0
0	3	0	1	5	0	3	0	3	0
0	2	5	2	4	3	0	3	0	0
0	2	4	1	0	6	1	4	1	0
0	3	0	1	5	0	3	0	2	0
0	2	6	2	4	3	2	3	0	0
0	2	4	1	0	6	2	1	1	0
0	2	6	2	4	4	0	3	6	0
0	2	4	1	0	6	1	6	1	0
0	0	0	0	0	0	0	0	0	0

f(m,n)

-1	0	1

0	-2	5	-1	-2	0	0	0
5	0	-1	1	-4	0	0	-3
4	-1	-4	5	1	-2	0	-4
0	-2	5	-1	-2	0	-1	0
6	0	-2	1	-2	0	-2	-3
4	-1	-4	5	2	-5	-1	-1
6	0	-2	2	-4	-1	6	-3
4	-1	-4	5	1	0	0	-6

f(m,n) *h(m)

2-D Convolution : separable filter

- If h(m,n) = h(m) * h(n), h(m,n) is separable

h(m,n)

1	0	-1
2	0	-2
1	0	-1

1	0	-1

1
2
1

h(-s)

-1	0	1

h(-t)

1
2
1

0	0	0	0	0	0	0	0	0	0
0	0	-2	5	-1	-2	0	0	0	0
0	5	0	-1	1	-4	0	0	-3	0
0	4	-1	-4	5	1	-2	0	-4	0
0	0	-2	5	-1	-2	0	-1	0	0
0	6	0	-2	1	-2	0	-2	-3	0
0	4	-1	-4	5	2	-5	-1	-1	0
0	6	0	-2	2	-4	-1	6	-3	0
0	4	-1	-4	5	1	0	0	-6	0
0	0	0	0	0	0	0	0	0	0

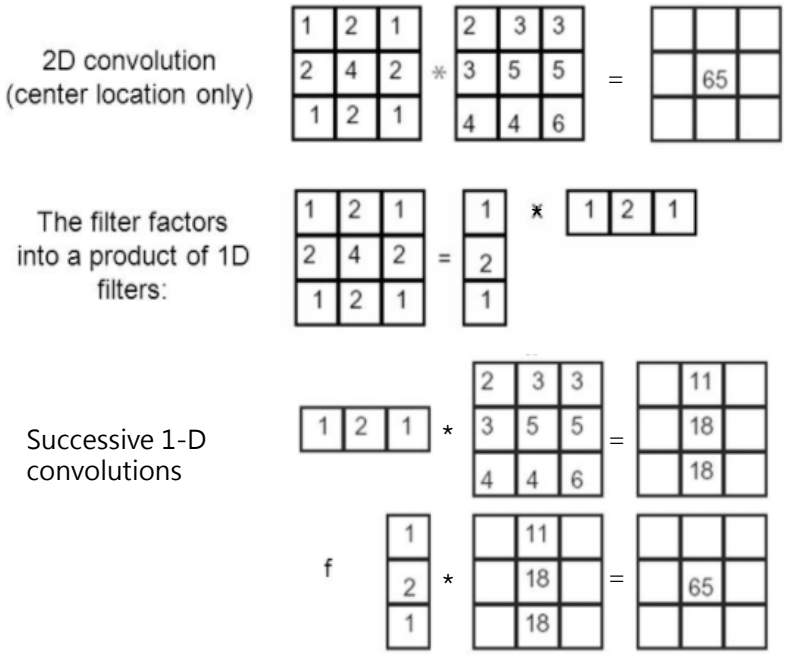
f(m,n) *h(m)

1
2
1

5	-4	9	-1	-8	0	0	-3
14	-3	-1	6	-9	-2	0	-10
13	-4	-4	10	-4	-4	-1	-11
10	-5	4	4	-5	-2	-4	-7
16	-3	-3	6	-4	-5	-6	-7
20	-2	-12	13	-2	-11	1	-8
20	-2	-12	14	-5	-7	11	-13
14	-2	-10	12	-2	-1	6	-15

[f(m,n) *h(m)]*h(n)

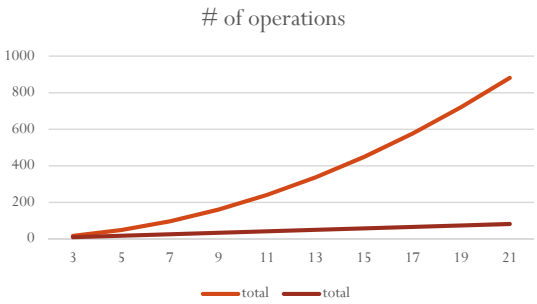
2-D Convolution : separable filter



Standard 2D convolution vs. Separable implement

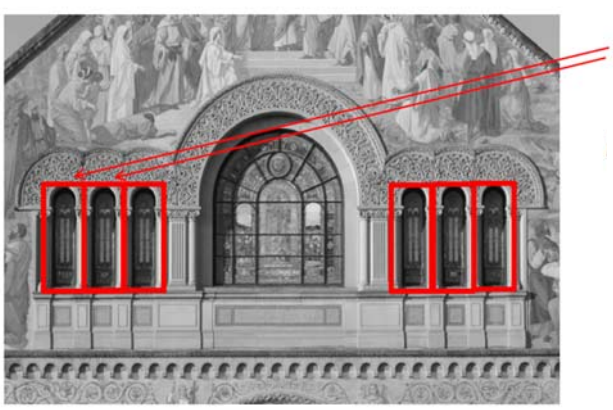
- **For an n×n size filter,**
- The standard 2D convolution implementation requires n^2 multiplications and $n^2 - 1$ additions per output pixel,
- whereas the spatially separable implementation significantly reduces this to $2n$ multiplications and $2(n-1)$ additions.

	Standard 2D convolution			Separable implementation		
n	mult.	add	total	mult.	add	total
3	9	8	17	6	4	10
5	25	24	49	10	8	18
7	49	48	97	14	12	26
9	81	80	161	18	16	34
11	121	120	241	22	20	42
13	169	168	337	26	24	50
15	225	224	449	30	28	58
17	289	288	577	34	32	66
19	361	360	721	38	36	74
21	441	440	881	42	40	82



Template Matching

- Problem: locate an object, described by template $t(m,n)$, in the image $f(m,n)$.



$t(m,n)$

$f(m,n)$

- Search for the best match by minimizing mean-squared error

$$E(i,j) = \sum_m \sum_n \left(f(m,n) - t(m-i,n-j)\right)^2$$ maximizing

$$E(i,j) = \sum_m \sum_n \left(f^2(m,n) + t^2(m-i,n-j) - 2f(m,n)t(m-i,n-j)\right)$$

- maximizing

$R_{tf}(i,j) = \sum_m \sum_n f(m,n)t(m-i,n-j) = f(m,n) \otimes t(m,n)$: Cross correlation

Convolution vs. Correlation

- Convolution

$$y(i,j) = f(m,n) * h(m,n) = \sum_m \sum_n f(m,n)h(i-m, j-n)$$

- Correlation

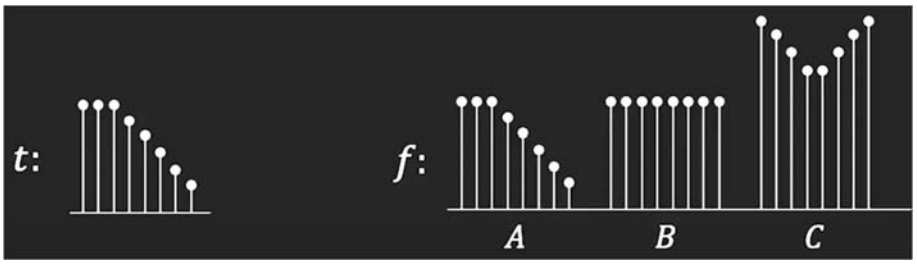
$$R_{tf}(i,j) = f(m,n) \otimes t(m,n) = \sum_m \sum_n f(m,n)t(m-i,n-j)$$

No flipping in Correlation

Problem with Correlation

- Correlation

$$R_{tf}(i,j) = f(m,n) \otimes t(m,n) = \sum_m \sum_n f(m,n)t(m-i,n-j)$$



$$R_{tf}(A) < R_{tf}(B) < R_{tf}(C)$$

- Normalized Correlation

$$N_{tf}(i,j) = \frac{\sum_m \sum_n f(m,n)t(m-i,n-j)}{\sqrt{\sum_m \sum_n f^2(m,n)}\sqrt{\sum_m \sum_n t^2(m-i,n-j)}}$$

$$N_{tf}(A) > N_{tf}(B) > N_{tf}(C)$$

Template Matching

- Normalized Correlation

$$N_{tf}(i,j) = \frac{\sum_m \sum_n f(m,n)t(m-i,n-j)}{\sqrt{\sum_m \sum_n f^2(m,n)}\sqrt{\sum_m \sum_n t^2(m-i,n-j)}}$$

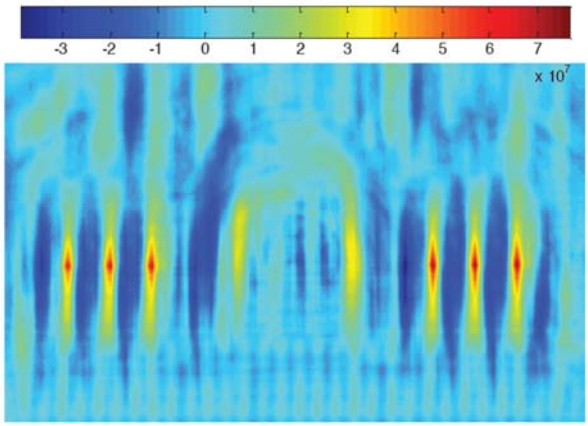
$$N_{tf}(A) > N_{tf}(B) > N_{tf}(C)$$



$t(m,n)$

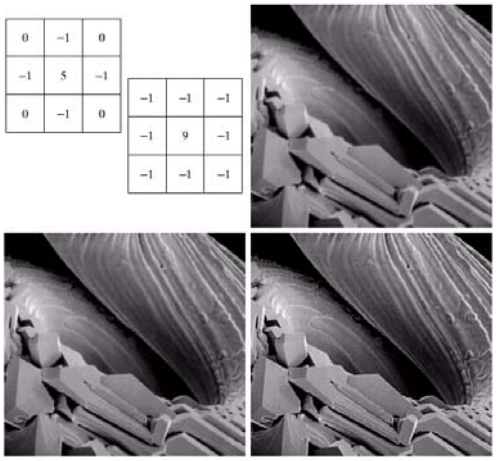


$f(m,n)$



$N_{tf}(i,j)$

Thank You
Next : Filtering



a b c
d e

FIGURE 3.41 (a) Composite Laplacian mask. (b) A second composite mask. (c) Scanning electron microscope image. (d) and (e) Results of filtering with the masks in (a) and (b), respectively. Note how much sharper (e) is than (d). (Original image courtesy of Mr. Michael Shaffer, Department of Geological Sciences, University of Oregon, Eugene.)